PLATFORM BUILT IN

lifepal

PRESENTED TO

ABOUND

PLATFORM · DATA ENGINEER

DATA ENGINEERING · FIRESIDE CHAT · NEXUS BEST PRACTICES FROM LIFEPAL

Nexus

Every change in Aurora, in the lake exactly once, proven against the source

Four years owning the data lake and pipelines behind risk and fraud models at Lifepal, a regulated insurance marketplace: CDC from PostgreSQL and MySQL into an S3 lake with AWS DMS and Debezium, Spark and Glue jobs that merge changes idempotently, and reporting, ad-hoc analytics and model training moved off the production database. This pack applies those practices to Abound's stack, Aurora → DMS → Glue → Iceberg → Athena, in a runnable repository with synthetic data. Every run figure here comes from that run; every Lifepal figure is from my CV.

PRESENTED BY

R. Firman Insan Muhammad (Vim)
Data Engineer · London

TO

Platform team
Abound

ROLE

Data Engineer
London hybrid

STACK

AWS DMS · Glue · Spark
Iceberg · Athena · Aurora

CONTENTS

What I'd like to cover

Fifteen minutes on what I've built, then the parts of the role I'd own first. Every page points to working code.

01	**Measured outcomes**	What changed at Lifepal, in numbers
02	**How my experience maps to the role**	Line by line against the job description
03	**The path**	Capture, land, apply, prove, serve
04	**Six CDC problems, and the code**	What DMS really delivers; bronze and silver MERGE
05	**Proving the lake against Aurora**	The reconciliation that caught a DMS gap, and its code
06	**Table design and platform cost**	Layout, its code, and where the AWS bill goes
07	**The bridge to Data Science**	Calibration on the lake; a point-in-time feature in SQL
08	**Governance and erasure**	Quality rules, tombstones, snapshot expiry
09	**On AWS**	DMS, the Glue 5.0 job, Lambda and ECS on Fargate
10	**My first 90 days, and a summary**	Moving workloads into the Data Lake

What changed at Lifepal

Figures from my CV and production work, described at the level I can share publicly.

99.7%
reporting latency cut: 24 h → under 5 min with CDC

75%
nightly batch cut: 6 h → 90 min by redesigning the layout

$11k
monthly platform cost against a $19k projection (≈42% under)

−60%
ad-hoc requests after self-serve datasets on the lake

−60%
data-trust escalations, with 500+ quality checks in CI

How: DMS and Kafka + Debezium from PostgreSQL and MySQL into S3; Spark and Glue jobs merging idempotently; partition pruning, Z-ordering and compaction; feature pipelines for LightGBM and XGBoost risk and fraud models.

Team: I mentor 3–5 engineers; two were promoted to Senior. Blameless post-mortems that end in durable fixes.

How my experience maps to the role

The job description's asks, what I've done, and where it shows in this repository.

THE ROLE ASKS FOR	WHAT I'VE DONE	HERE
Own the Data Lake and its ETL	Owned the S3 lake and pipelines end to end at Lifepal	src/nexus
CDC, ideally AWS DMS	DMS and Debezium from PostgreSQL and MySQL, merged idempotently	bronze.py · silver.py
Spark and Glue; RDS workloads into pipelines	Moved reporting, analytics and training off RDS	glue/cdc_job.py
Strong SQL on Aurora PostgreSQL / MySQL	Query tuning and indexing on transactional RDS	checks.py
Data structures and clustering	Partitioning, Z-order, compaction: 6 h → 90 min	layout.py
Lower AWS cost	$11k a month against a $19k projection	layout.py
Bridge to Data Science	Point-in-time training sets, parity, drift, recalibration	gold.py
Governance and standards	500+ checks in CI, PII masking, RBAC, lineage	contracts/
SOLID in practice	Connectors behind common interfaces, config injected, PyTest	src/nexus · tests/
Iceberg	Delta in production; Iceberg in this repo, one to one	silver.py · layout.py

Aurora to the lake, proven

Five steps from a commit in the lending database to a table a data scientist can trust. The run: a full load, then three months of CDC in 6 batches by arrival time.

01 · CAPTURE
DMS
Full load, then CDC: Op, cdc_ts and AR_H_CHANGE_SEQ on every row.

→

02 · LAND
Bronze
Every change keyed on (key, change_seq). A full replay changed 0 rows.

→

03 · APPLY
MERGE
Newest change_seq wins; deletes are tombstones; a reload is authoritative.

→

04 · PROVE
18/18
Row by row against Aurora; quality rules; freshness; erasure.

→

05 · SERVE
Iceberg
Partitioned and sorted for Athena; point in time for calibration.

THE RUN **59,754** source changes **417** DMS files **57,025** changes re-read on replay, **0** new **16** rows caught by reconciliation **195s** end to end

What DMS really delivers

Each is planted in the synthetic data, handled by the pipeline and covered by a test.

PROBLEM	IN THE RUN	WHAT HANDLES IT
Re-sent after a task restart	14 changes re-sent in a new file	Bronze keys on (key, change_seq): 14 skipped
A file delivered late	customers file for 2026-09-03 landed 2026-09-18	List by arrival; order by change_seq
A delete, then an older update	C00016 erased; an earlier update arrived after	Tombstone: 0 live rows after
A column added mid-stream	ALTER TABLE loans ADD COLUMN apr_bps	ADD COLUMNS in bronze, metadata only
Two changes, one timestamp	53 late payments, two statements per transaction	AR_H_CHANGE_SEQ orders them
Changes DMS never wrote	16 repayment changes, 20 Sep 02:00–04:00	Reconciliation to source, then a reload

Safe re-runs: bronze keyed on the change

Every DMS file can arrive twice, late or out of order. Bronze keeps one row per change, so a re-run or a re-send changes nothing.

$$\text{ingest}(\text{ingest}(B, F), F) = \text{ingest}(B, F) \cdot \text{identity} = (\text{key}, \text{change_seq})$$

SRC/NEXUS/BRONZE.PY — PYSPARK · ICEBERG 1.7

```python
def ingest(spark: SparkSession, src: dict, files: list[Path], batch_id: int) -> dict:
    name, pk = bronze_name(src), src["primary_key"]
    log = table("ops", "ingested_files")
    if not files:
        return {"files": 0, "rows_read": 0, "new_changes": 0, "duplicates": 0,
"columns_added": []}
    df = (read_files(spark, files)
          .withColumn("cdc_ts", F.to_timestamp("cdc_ts"))
          .withColumn("_batch_id", F.lit(batch_id))
          .withColumn("_ingested_at", F.current_timestamp()))
    rows_read = df.count()
    # one copy of each change inside the batch; MERGE then skips any copy bronze already
has
    w = Window.partitionBy(*pk, "change_seq").orderBy("_source_file")
    df = df.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1").drop("_rn")

    if not spark.catalog.tableExists(name):
        df.limit(0).writeTo(name).using("iceberg").partitionedBy(F.days("cdc_ts")) \
            .tableProperty("format-version", "2").create()
    added = _evolve(spark, name, df)
    before = spark.table(name).count()
    df.createOrReplaceTempView("batch")
    on = " AND ".join(f"t.`{k}` = s.`{k}`" for k in pk + ["change_seq"])
    spark.sql(f"MERGE INTO {name} t USING batch s ON {on} WHEN NOT MATCHED THEN INSERT *")
    new = spark.table(name).count() - before
```

THE GUARANTEE

- Each change is identified by its primary key and **AR_H_CHANGE_SEQ**, which DMS makes unique and increasing. A file name is not an identity: after a restart DMS re-sends the same changes in a **new** file.
- At-least-once delivery from DMS becomes an exactly-once effect on the table.

AT LIFEPAL

- CDC from PostgreSQL and MySQL with DMS and with Kafka + Debezium, merged idempotently by Spark and Glue jobs. Reporting latency fell from 24 hours to under 5 minutes.

IN THE RUN

- DMS re-sent **14** changes after a restart: skipped. Replaying every file DMS ever wrote, **57,025** changes, added **0** rows.
- A column added mid-stream (**apr_bps**) becomes ALTER TABLE ADD COLUMNS: metadata only, no rewrite.

The newest change wins. Deletes stay deleted.

Silver holds the current state of every Aurora table. Three rules make it right whatever order the changes arrive in.

SRC/NEXUS/SILVER.PY — PYSPARK · ICEBERG MERGE

```
def latest(df: DataFrame, src: dict) -> DataFrame:
    w = Window.partitionBy(*src["primary_key"]).orderBy(F.col("change_seq").desc())
    df = df.withColumn("_rn", F.row_number().over(w)).filter("_rn = 1").drop("_rn")
    deleted = F.col("Op") == "D"
    keep = set(src["primary_key"])
    return df.select(*[F.col(c) if c in keep else F.when(deleted,
F.lit(None)).otherwise(F.col(c)).alias(c)
                       for c in src["columns"]],
                     deleted.alias("is_deleted"), "cdc_ts", "change_seq")

    on = " AND ".join(f"t.`{k}` = s.`{k}`" for k in src["primary_key"])
    by_source = (" WHEN NOT MATCHED BY SOURCE AND NOT t.is_deleted THEN UPDATE SET
t.is_deleted = true"
                 if authoritative else "")
    spark.sql(f"""MERGE INTO {name} t USING silver_batch s ON {on}
                  WHEN MATCHED AND s.change_seq > t.change_seq THEN UPDATE SET *
                  WHEN NOT MATCHED THEN INSERT *{by_source}""")
```

THREE RULES

- **change_seq decides.** Two statements in one transaction share a cdc_ts; only the sequence orders them.
- **Deletes are tombstones** that keep their change_seq, so an older update arriving late can't resurrect the row. Non-key values are nulled: erasure means erasure.
- **A reload is authoritative:** WHEN NOT MATCHED BY SOURCE tombstones what the source no longer has.

IN THE RUN

- 53 two-statement transactions ordered correctly; **C00016**'s late update arrived after the delete and left **0** live rows.

A check failed. The run fixed it.

No key can catch a row that never arrived. Only a comparison with the source can.

16 repayments rows didn't match Aurora

DMS delivered no repayments changes between 03:00 and 04:00 on 2026-09-20, the hour of the daily direct-debit run. The source has them, so the task resumed from the wrong position after an interruption: the rows are missing, not late.

Bronze had **0** changes in that hour, against a median of **10** on other days.

Fix: Reload repayments from the source (DMS 'reload table'), apply it as authoritative, re-run the reconciliation. Then alert on hourly change volume per table so a gap pages before a month-end report finds it.

SRC/NEXUS/SILVER.PY — A RELOAD IS AUTHORITATIVE

```
MERGE INTO lake.silver.repayments t
USING silver_batch s ON t.repayment_id = s.repayment_id
WHEN MATCHED AND s.change_seq > t.change_seq
  THEN UPDATE SET *
WHEN NOT MATCHED THEN INSERT *
WHEN NOT MATCHED BY SOURCE AND NOT t.is_deleted
  THEN UPDATE SET t.is_deleted = true
```

After the reload: **18/18** checks pass, including reconciliation on every table.

Reconciliation: the check no key can replace

Silver against the source's own rows, on every contract column. It finds what a CDC stream can't tell you: the changes that never came.

SRC/NEXUS/CHECKS.PY — PYSPARK

```pyspark
def reconcile(spark: SparkSession, src: dict) -> dict:
    pk, cols = src["primary_key"], list(src["columns"])
    lake = current(spark, src).select(*cols).alias("l")
    srcd = source_rows(spark, src).alias("s")
    j = lake.join(srcd, [lake[k] == srcd[k] for k in pk], "full_outer")
    in_lake, in_src = F.col(f"l.{pk[0]}").isNotNull(), F.col(f"s.{pk[0]}").isNotNull()
    same = F.lit(True)
    for c in cols:
        same = same & F.col(f"l.{c}").eqNullSafe(F.col(f"s.{c}"))
    j = j.select(F.coalesce(F.col(f"l.{pk[0]}"), F.col(f"s.{pk[0]}")).alias("key"),
                 F.when(~in_lake, "missing_in_lake").when(~in_src, "extra_in_lake")
                  .when(~same, "different").otherwise("match").alias("result"),
                 *[F.col(f"l.{c}").alias(f"lake_{c}") for c in cols[1:]],
                 *[F.col(f"s.{c}").alias(f"source_{c}") for c in cols[1:]])
```

WHY IT EXISTS

- Duplicates and late files are visible in the stream. A **missing** change isn't: keys look fine, counts look fine, and a month-end report is quietly wrong.

IN THE RUN

- DMS skipped **16** repayment changes on 20 September. Reconciliation found **16** rows that differ; the investigation showed **0** bronze changes in an hour that normally has **10**.
- Reload, apply as authoritative, check again: **18/18** pass.

ON AWS

- Run it nightly against an Aurora **read replica**, never the writer, and alert on hourly change volume per table so a gap pages before a report finds it.

Read only what you need

Athena bills bytes scanned at $5 per TB. The same queries, planned from Iceberg's own file statistics on three layouts.

QUERY	NAIVE (UNPARTITIONED)	MONTHS(BOOKED_AT)	+ SORTED, COMPACTED	LESS SCANNED
Data Science: one customer's last 90 days	578 KB · 72/221 files	254 KB · 5/18	55 KB · 3/70	90%
Reporting: every transaction in September	237 KB · 28/221 files	52 KB · 2/18	54 KB · 3/70	77%

Hidden partitioning

months(booked_at): queries filter on the column, and the spec can evolve without a rewrite.

Sort order

WRITE ORDERED BY customer_id, booked_at: min/max bounds prune files for per-customer reads.

Maintenance

rewrite_data_files after MERGE; expire_snapshots and remove_orphan_files keep storage honest.

At Lifepal the same ideas, on Delta (partition pruning, Z-ordering, compaction, AQE), cut the nightly batch from 6 h to 90 min.

Partition, sort, compact

The layout is part of the table's definition, not a tuning afterthought, and maintenance runs on a schedule.

SRC/NEXUS/SILVER.PY · SRC/NEXUS/LAYOUT.PY — ICEBERG 1.7

```
def _create(spark: SparkSession, name: str, df: DataFrame, src: dict) -> None:
    w = df.limit(0).writeTo(name).using("iceberg").tableProperty("format-version", "2")
    layout = src.get("layout")
    if layout:
        col = layout["partition_by"].split("(")[1].rstrip(")")
        w = w.partitionedBy(F.months(col))
    w.create()
    if layout:
        spark.sql(f"ALTER TABLE {name} WRITE ORDERED BY {', '.join(layout['sort_by'])}")

def compact(spark: SparkSession) -> dict:
    """Rewrite the small files MERGE leaves behind into sorted ones, then drop what no
snapshot needs."""
    name = "silver.ob_transactions"
    res = spark.sql(f"""CALL {CATALOG}.system.rewrite_data_files(table => '{name}',
strategy => 'sort',
                        sort_order => 'customer_id ASC NULLS LAST, booked_at ASC NULLS
LAST',
                        options => map('rewrite-all', 'true', 'target-file-size-bytes',
'{TARGET_FILE_BYTES}'))""").first()
    return {"rewritten_files": res.rewritten_data_files_count, "added_files":
res.added_data_files_count}
```

WHAT EACH DOES

- **months(booked_at)**: a hidden partition transform. Queries filter on booked_at; the spec can evolve without rewriting data.
- **WRITE ORDERED BY customer_id**: each file covers a narrow range of customers, so min/max bounds skip files.
- **rewrite_data_files**: MERGE and streaming leave small files; compaction puts them back in order.

MEASURED HERE

- Per-customer read: **90%** less scanned. Monthly report: **77%** less. Same answers on every layout.

AT LIFEPAL

- The same ideas on Delta (partition pruning, Z-ordering, compaction, AQE) cut the nightly batch from 6 h to 90 min, with the platform at $11k a month against a $19k projection.

Managing platform cost

Cost is a defect like any other: attributed to a team, visible every week, and fixed where it starts.

INFRA/FINOPS/COST_BY_TEAM.SQL · ATHENA OVER THE CUR · INFRA/TERRAFORM/MAIN.TF · TERRAFORM

```sql
SELECT resource_tags['user_owning_team']            AS owning_team,
       resource_tags['user_workload']               AS workload,
       product['product_name']                      AS service,
       round(sum(line_item_unblended_cost), 0)      AS spend_90d,
       round(100 * sum(line_item_unblended_cost)
             / sum(sum(line_item_unblended_cost)) OVER (), 1) AS pct
FROM   cur.data_exports
WHERE  line_item_usage_start_date >= current_date - INTERVAL '90' DAY
  AND  product['product_name'] IN ('AWS Glue', 'Amazon Athena', 'Amazon Simple Storage Service',
                                   'AWS Database Migration Service', 'Amazon Relational Database Service')
GROUP BY 1, 2, 3
ORDER BY spend_90d DESC;
```

```hcl
resource "aws_athena_workgroup" "audience" {
  for_each = { data_science = 200, finance = 50, dashboards = 20 }   # GB a single query may scan
  name     = each.key
  tags     = merge(local.tags, { workload = each.key })
  configuration {
    enforce_workgroup_configuration    = true
    publish_cloudwatch_metrics_enabled = true
    bytes_scanned_cutoff_per_query     = each.value * 1024 * 1024 * 1024
    result_configuration {
      output_location = "s3://${aws_s3_bucket.lake.bucket}/athena-results/${each.key}/"
    }
  }
}
```

FOUR MECHANISMS

- **Attribution.** Every resource carries an owning team and a workload tag; the Cost and Usage Report, queried in Athena, says who spends what.
- **Guard-rails.** One Athena workgroup per audience with a per-query scan cap, so a careless dashboard can't scan the lake on every refresh.
- **Right-sized compute.** Glue auto-scaling for the hourly apply; FLEX execution for backfills and maintenance.
- **A budget alert acted on weekly,** in the same forum as incidents.

AT LIFEPAL

- The platform ran at about **$11k a month against a $19k projection**. Most of it came from layout (partition pruning, Z-ordering, compaction) and from refreshing data only as often as people need it.

Where the bill goes, and the lever for each

On Abound's stack the services are few and the levers are known. Measure first, fix the biggest line, prove nothing broke.

SERVICE	WHAT YOU PAY FOR	THE LEVER
Athena	bytes scanned, $5 per TB	Partition and sort; compact; pre-aggregate for dashboards; workgroup scan caps. Here: 90% less per customer read
Glue	DPU-hours	Only new files (the file log), auto-scaling, FLEX for non-urgent jobs, no hourly run that nobody reads hourly
S3	storage and requests	expire_snapshots and remove_orphan_files weekly; landing files expire after 30 days; Intelligent-Tiering for cold data
DMS	replication instance hours	Right-size the instance; replicate only the tables and columns that are used
Aurora	instance size and replicas	Move analytics reads to the lake, so the writer never grows for reporting load

1 · Measure

Spend by team and workload from the CUR; Athena bytes by workgroup from CloudTrail (infra/finops/).

2 · Fix the biggest line

Usually small files and full scans in Athena, or over-provisioned, over-frequent Glue jobs.

3 · Prove nothing broke

Every saving ships with the reconciliation and freshness checks green: cheaper, never less correct.

Calibrate on the lake, as of the decision

Calibration needs the score at decision time, what happened next, and every feature as it was known then.

SCORE BAND	LOANS	PREDICTED	OBSERVED
0–10%	59	7%	8%
10–20%	164	15%	19%
20%+	98	25%	30%

gold.ds_calibration_by_band: the scorecard is too optimistic above 10%, and that gap is what recalibration fixes.

Point in time, or a leak

Open Banking enrichment relabels salary credits days after the fetch. A feature built from today's labels differs for **72 of 111** decisions, adding **£2,807** of income on average that the model never saw.

At Lifepal: feature pipelines behind the customer-risk and claims-fraud models: point-in-time training sets, training/serving parity, drift monitoring and recalibration.

A feature as it was known at decision time

Open Banking enrichment changes rows after the fetch. The change history in silver lets a feature see exactly what the model saw.

SRC/NEXUS/GOLD.PY — SPARK SQL · ICEBERG

```
        pit AS (
          SELECT a.application_id, sum(h.amount) AS income_90d
          FROM apps a JOIN {table('silver', 'ob_transactions_history')} h
            ON h.customer_id = a.customer_id AND h.booked_at >= a.decided_at - INTERVAL 90
DAYS
           AND h.booked_at < a.decided_at AND h.fetched_at <= a.decided_at
           AND h.valid_from <= a.decided_at AND a.decided_at < h.valid_to AND NOT
h.is_deleted
          WHERE h.category = 'salary' GROUP BY 1),
        naive AS (
          SELECT a.application_id, sum(t.amount) AS income_90d
          FROM apps a JOIN {table('silver', 'ob_transactions')} t
            ON t.customer_id = a.customer_id AND t.booked_at >= a.decided_at - INTERVAL 90
DAYS
           AND t.booked_at < a.decided_at
          WHERE NOT t.is_deleted AND t.category = 'salary' GROUP BY 1)
```

THE TRAP

- A join on today's rows uses today's categories. Salary credits relabelled after the decision inflate income the model never saw, so offline results look better than production.

IN THE RUN

- The naive feature differs for **72 of 111** decisions since 1 July, by **£2,807** of income on average.

AT LIFEPAL

- Feature pipelines behind the customer-risk and claims-fraud models (LightGBM, XGBoost): point-in-time training sets, training/serving parity, drift monitoring and recalibration.

Standards that run as code

Quality rules from the contract, reconciliation, freshness and erasure: checked on every run, failing loudly.

Contracts

contracts/sources.yml: keys, types, PII columns, what to reconcile. quality.yml: the rules on silver.

Erasure

Deletes become tombstones with values nulled; bronze is purged; expire_snapshots on 10 tables so time travel can't recover PII.

Freshness

Measured on a table that changes every day; in production, the DMS heartbeat and CDCLatencySource.

At Lifepal: 500+ quality checks in CI, freshness and volume monitoring with alerting, PII masking, role-based access and lineage. Data-trust escalations fell by 60%.

The same code, deployed

Written to be read and reviewed; not applied, since there's no AWS account behind a portfolio.

CAPTURE
infra/dms/
S3 target: CSV with header, TimestampColumnName=cdc_ts, AR_H_CHANGE_SEQ → change_seq.

APPLY
glue/cdc_job.py
Glue 5.0 (Spark 3.5, Iceberg 1.7) with the Glue Data Catalog; the same bronze and silver code.

AROUND IT
Lambda · ECS on Fargate
Lambda on S3 events and failed checks; Fargate tasks for reloads, backfills and the reconciliation export.

PROVISION
infra/terraform/
KMS, S3, DMS instance, endpoints and task, Glue job and schedule, least-privilege IAM.

The same code, as a Glue job

The only thing the job adds is the catalog: everything else is the Python that runs locally and in CI.

GLUE/CDC_JOB.PY — AWS GLUE 5.0 · SPARK 3.5 · ICEBERG 1.7

```
spark = (SparkSession.builder
         .config("spark.sql.extensions",
"org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions")
         .config("spark.sql.catalog.glue_catalog", "org.apache.iceberg.spark.SparkCatalog")
         .config("spark.sql.catalog.glue_catalog.catalog-impl",
"org.apache.iceberg.aws.glue.GlueCatalog")
         .config("spark.sql.catalog.glue_catalog.io-impl",
"org.apache.iceberg.aws.s3.S3FileIO")
         .config("spark.sql.catalog.glue_catalog.warehouse", args["warehouse"])
         .getOrCreate())
glue = GlueContext(SparkContext.getOrCreate())
job = Job(glue)
job.init(args["JOB_NAME"], args)

from nexus import bronze, checks, silver  # noqa: E402  (after NEXUS_CATALOG is set)
from nexus.contracts import load_sources  # noqa: E402

until = datetime.now(timezone.utc).replace(tzinfo=None)
batch_id = int(until.strftime("%Y%m%d%H"))
failed = []
for src in load_sources():
    # on S3, bronze.list_arrivals is an s3 list filtered on LastModified; the file log
makes it incremental
    files = bronze.new_files(spark, src, until)
    bronze.ingest(spark, src, files, batch_id)
    silver.apply(spark, src, batch_id)
    failed += [c for c in checks.quality(spark, src) if c["status"] == "fail"]
```

DMS SETTINGS IT EXPECTS

- CSV with a header, **TimestampColumnName=cdc_ts**, Op on the full load too, and **$AR_H_CHANGE_SEQ** added as change_seq by a table-mapping rule (infra/dms/).

AROUND IT

- An hourly Glue trigger. A Lambda posts failing checks to on-call; reloads and backfills run as one-off ECS tasks on Fargate. Terraform for all of it, least-privilege IAM, KMS on the lake.

STATED PLAINLY

- Written to be read and reviewed; not applied, since there is no AWS account behind a portfolio. The same functions ran for every number in this pack.

Moving workloads into the Data Lake

The role's first 6–12 months, as I'd start them.

Days 1–30 · Listen and ship

Learn the lake, the DMS tasks and the models from their owners. A small fix in production early.

Days 31–60 · Prove it

Row-by-row reconciliation from Aurora for the tables Data Science uses; a change-volume alert per table.

Days 61–90 · Move one workload

Model calibration on the lake, point in time, with Data Science; the RDS query switched off.

Day 90 · Measured

Fresher data, a smaller Athena bill per query, and one fewer heavy read on Aurora.

PROVEN AT LIFEPAL **24h → <5m** reporting latency · **6h → 90m** nightly batch · **$11k** vs $19k projection · **−60%** ad-hoc requests · **500+** checks in CI · **−60%** data-trust escalations

Everything in this pack runs at **rfim.github.io/nexus-abound**. Synthetic data; an independent application, not affiliated with or endorsed by Abound.

Summary

Thank you for reading. Every Lifepal figure is from my CV and production work, and every run figure from the code; I'm happy to go as deep as you like on any of them.

CDC you can trust

DMS changes keyed on change_seq; a full replay changed 0 rows; tombstones that stay deleted.

Proven against Aurora

Row-by-row reconciliation caught 16 rows DMS never sent; a reload fixed them.

Built for Data Science

Calibration by score band and features as of the decision, from the change history.

Cheaper by design

Partitioned, sorted, compacted: 90% less scanned. At Lifepal, $11k against $19k.

Governed end to end

Contracts, quality rules, erasure across bronze, silver and every snapshot.

Production experience and working knowledge

In production: Python, SQL, PySpark, AWS (S3, DMS, Glue, Lambda, ECS), Aurora / RDS PostgreSQL and MySQL, Kafka + Debezium CDC, Delta Lake on Databricks, PyTest and GitLab CI, Terraform, Docker. dbt and Snowflake at Deloitte.

Working level: Athena. This repository is my Iceberg work: MERGE, hidden partitioning, sort orders, compaction and snapshot expiry on Spark 3.5, as Glue 5.0 runs it.

In one line

I get every change out of a transactional database reliably, prove the lake matches it, and lay it out so Data Science and the business can read it fast and cheaply. I've done it in regulated financial services, and I'd like to do it on Abound's lake.

lifepal **Abound**

R. Firman Insan Muhammad (Vim) · Data Engineer · London · firman.insan.m@gmail.com · +44 747 061 4781 · right to work in the UK, no sponsorship required · code and live site: github.com/rfim/nexus-abound